

LC 928: Malware Spread 2 (Dropbox)

1. Intuition & Problem Breakdown

It's different from **LC 924**, because:

- **LC 924**: Infection spreads *inside* the connected component.
- **LC 928**: Infection spreads *through edges* after removing one infected node.

Example: 0 - 2 - 3 and (1)

Now if we remove 0, 1 can't reach 2 and 3, so 1 is the answer.

2. The Core Logic

So what I have figured out is: **Saving depends on which clean nodes can be reached by which infected nodes.**

Therefore, we use: **DSU + Graph Traversal.**

Let (1) and (3) be infected. Let's say Component A is reachable from both (1) and (3). Then removing *one* won't help.

Say Component A is reachable *only* from (3). Then removing 3 saves A.

3. Algorithm Steps

- **Step 1:** Build DSU only on clean nodes. Ignore the infected nodes; we are just creating components.
- **Step 2:** For every infected node, look at its neighbours. For every clean component, we have to store which infected nodes can directly reach it.
i.e. leader → { infected nodes }
- **Step 3:** We choose the node saving the max vertices.

4. C++ Implementation

```
#include<bits/stdc++.h>
using namespace std;

class dsu{
public:

    vector<int> parent;
    vector<int> size;
    dsu(int n){
        parent.resize(n);
        size.resize(n,1);

        for(int i=0;i<n;i++){
            parent[i]=i;
        }
    }

    int findparent(int node){

        if(parent[node]==node){
            return node;
        }

        parent[node]=findparent(parent[node]);
        return parent[node];
    }

    void unionit(int u,int v){
        int pu=findparent(u);
        int pv=findparent(v);
```

```

        if(pu==pv) return;

        if(size[pu]<size[pv]) swap(pu,pv);

        parent[pv]=pu;
        size[pu]+=size[pv];
    }
};

class Solution {
public:
    int minMalwareSpread(vector<vector<int>>& graph, vector<int>& initial)
    {

        int n=graph.size();
        dsu ds(n);

        unordered_set<int> infected(initial.begin(),initial.end());
        //sirf o(1) lookup ke liye bnaya
        //dsu sirf clean node pe krenge
        //infected avoid

        for(int i=0;i<n;i++){
            if(infected.count(i)) continue;
            for(int j=i+1;j<n;j++){
                if(infected.count(j)) continue;

                if(graph[i][j]){
                    ds.unionit(i,j);
                }
            }
        }

        //har component ka leader store krke rkhega
        //ki ye kis kis infected se bimaar he

        unordered_map<int,unordered_set<int>> touch;
        for(auto &node:initial){ //har infected har direction me ghumega to
look for clean comp
            for(int nei=0;nei<n;nei++){

                if(graph[node][nei]==0) continue; //connection hi nhi
                if(infected.count(nei)) continue; //neighbour khud infected
he

                int boss=ds.findparent(nei);

```

```

        touch[boss].insert(node);
    }
}

unordered_map<int,int> saves;
//har removal kitno ko bachayega

for(auto &it:touch){

    int boss=it.first;
    if(it.second.size()==1) {
        //must be reachable by only one infected , otherwise
        //ek ko htayenge dusra remove kr dega
        //eventually 0 loss

        int node=*it.second.begin();
        saves[node]+=ds.size[boss];
    }
}

int ans=INT_MAX;
int maxu=-1;

for(auto &node:initial){

    if(saves[node]>maxu){
        maxu=saves[node];
        ans=node;
    }

    else if(saves[node]==maxu){
        ans=min(ans,node);
    }
}

return ans;

}

};

```